LAPORAN PROYEK PROJECT BASED LEARNING (PBL)

PENGEMBANGAN RESTFUL API SISTEM MANAJEMEN PENGELUARAN PERJALANAN (TRAVEL EXPENSE TRACKER) TERINTEGRASI KONVERSI KURS VALAS REAL-TIME DAN ANALISIS BATAS ANGGARAN

KELOMPOK 2

Disusun Oleh:

Lidiya (F5512530118)

Tabita Creaselia Belicia Theona (F5512520086)

Muhammad Zikri Adam (F5512530117)

Trista Sheryl Lambei (F5512530106)

PROGRAM STUDI S1 TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS TADULAKO

2026

2. DESKRIPSI APLIKASI

2.1 Latar Belakang dan Masalah yang Diselesaikan

Saat bepergian ke luar negeri atau melakukan perjalanan dinas lintas negara, kita kerap bertransaksi menggunakan berbagai mata uang asing seperti USD, JPY, EUR, SGD, maupun MYR. Masalah yang sering dihadapi adalah sulitnya mencatat dan mengontrol pengeluaran harian secara tepat waktu. Banyak orang mencatat transaksi secara manual di catatan ponsel hanya dengan nominal mata uang lokal negara tujuan, tanpa mengetahui secara pasti berapa nilai belanjaan tersebut jika dikonversikan ke Rupiah berdasarkan kurs riil saat transaksi berlangsung.

Akibatnya, pelancong sering salah memperkirakan sisa uang yang dimiliki hingga akhirnya anggaran perjalanan habis sebelum waktunya (over budget). Untuk mengatasi kendala tersebut, proyek ini mengembangkan aplikasi backend berbasis RESTful API menggunakan Node.js murni dengan nama Travel Expense Tracker. Aplikasi ini mencatat riwayat pengeluaran perjalanan ke dalam penyimpanan lokal, sekaligus mengintegrasikan data kurs valuta asing dari Public API secara real-time di sisi server untuk menghitung nilai konversi Rupiah dan menganalisis batas aman anggaran secara otomatis.

2.2 Sasaran Pengguna Aplikasi

Aplikasi berbasis console ini ditujukan untuk beberapa pengguna berikut:

1. Wisatawan Mandiri: Membantu mencatat biaya penginapan, tiket, kuliner, dan belanja oleh-oleh selama di luar negeri dengan konversi langsung ke Rupiah.
2. Mahasiswa Pertukaran Pelajar / Studi ke Luar Negeri: Mempermudah pengelolaan uang saku bulanan agar tidak melebihi anggaran yang telah disiapkan.
3. Pelaku Perjalanan Dinas: Membantu rekapitulasi pengeluaran operasional di lapangan dengan rujukan nilai kurs yang jelas dan transparan.

3. RANCANGAN API

3.1 Daftar Resource dan Atribut Data

Data utama yang dikelola sistem adalah resource 'expenses', yang menyimpan setiap transaksi pengeluaran. Seluruh data disimpan secara persisten ke dalam berkas JSON (data/expenses.json) menggunakan modul inti fs/promises agar data tetap tersimpan meskipun server dimatikan. Tabel berikut menjelaskan struktur field, tipe data, serta aturan validasinya:

Nama Field	**Tipe Data**	**Aturan Validasi**	**Keterangan**
id	Integer	Auto-increment, unik, read-only	Nomor identitas unik transaksi
title	String	Wajib, tidak boleh string kosong (min 1 char)	Nama aktivitas / belanja pengeluaran
category	String	Enum: akomodasi, transportasi, konsumsi, belanja, hiburan, lainnya	Klasifikasi pengeluaran
amount	Number (Float/Int)	Wajib berupa angka dan > 0	Nominal pengeluaran dalam mata uang asal
currency	String	Enum: USD, JPY, EUR, SGD, MYR, GBP, AUD, IDR	Kode mata uang asing (ISO 4217)
date	String	Format tanggal ISO: YYYY-MM-DD	Waktu transaksi dilakukan
notes	String	Opsional, teks bebas	Catatan penjelasan rinci transaksi

3.2 Tabel Endpoint RESTful API

Seluruh endpoint dibangun langsung menggunakan modul bawaan http pada Node.js tanpa menggunakan framework seperti Express. Penanganan kode status

HTTP disesuaikan dengan standar RESTful (200 untuk permintaan sukses, 201 untuk penambahan data baru, 400 untuk input tidak valid, 404 jika data tidak ditemukan, dan 405 jika metode HTTP tidak diizinkan):

Metode	**Endpoint URL**	**Query / Body Request**	**Status Code**	**Fungsi & Perilaku**
GET	/	-	200 OK	Mengembalikan identitas server, nama anggota kelompok, dan daftar route.
GET	/expenses	?category=... &currency=...	200 OK	Mengambil seluruh pengeluaran diperkaya konversi kurs live IDR, mendukung filter kategori & mata uang.
GET	/expenses/summary	?budget=... (opsional)	200 OK	Menghitung total IDR, sisa budget, persentase pemakaian, dan status evaluasi risiko anggaran.
GET	/expenses/:id	-	200 OK / 404 Not Found	Mengambil rincian 1 catatan transaksi beserta kurs per satuan unit saat ini.
POST	/expenses	JSON: title, category, amount, currency, date, notes	201 Created / 400 Bad Request	Menambahkan transaksi baru, memvalidasi input, menghitung ID baru, dan menyimpan ke file JSON.

PUT	/expenses/:id	JSON: field yang diperbarui	200 OK / 400 / 404	Memperbarui data transaksi secara parsial atau penuh dan menyimpannya kembali ke file JSON.
DELETE	/expenses/:id	-	200 OK / 404 Not Found	Menghapus catatan transaksi dari file JSON berdasarkan ID yang dituju.

4. INTEGRASI PUBLIC API

4.1 API yang Dipilih dan Alasan Pemilihan

Public API yang kami integrasikan ke dalam server adalah ExchangeRate-API (https://open.er-api.com/v6/latest/USD). Alasan pemilihan API ini antara lain: 1. Bebas Autentikasi: Endpoint publik ini dapat diakses langsung tanpa memerlukan API key, sehingga proses request dari server berjalan lancar tanpa kendala batas kuota akun.

2. Data Mata Uang Lengkap: Mendukung nilai tukar untuk lebih dari 160 mata uang dunia terhadap USD sebagai mata uang acuan, termasuk IDR, JPY, EUR, SGD, dan MYR.

3. Respon JSON Terstruktur dan Cepat: Waktu respon API cukup stabil dan format datanya mudah diproses oleh Node.js.

4.2 Field Data yang Diambil

Dari data respon JSON yang diterima dari ExchangeRate-API, server mengambil dan memproses beberapa bagian penting berikut:
- rates: Daftar rasio nilai tukar mata uang dunia, khususnya rates.IDR, rates.JPY, rates.EUR, rates.SGD, dan rates.MYR.

- time_last_update_utc: Waktu pembaruan kurs terakhir dari server penyedia data sebagai informasi pendukung.

- result: Status penanda keberhasilan respon dari API (bernilai 'success').

4.3 Cara Data Public API Diolah di Sisi Server

Sesuai ketentuan tugas, data kurs yang diperoleh dari Public API tidak sekadar diteruskan mentah ke client, tetapi diolah melalui logika perhitungan di server kelompok:

1. Konversi Nilai Transaksi ke Rupiah (IDR):

Karena kuotasi API berbasis USD, server mengonversi nominal mata uang asal ke USD terlebih dahulu, kemudian mengalikannya dengan kurs IDR:

Nominal IDR = Math.round((Nominal Asal / Rate Asal) * Rate IDR)

Kurs Acuan = Math.round((1 / Rate Asal) * Rate IDR)

Hasil konversi ini disematkan pada setiap data pengeluaran yang dikirimkan ke client.

2. Perhitungan Sisa Anggaran dan Evaluasi Batas Belanja (/expenses/summary):

Server menjumlahkan seluruh pengeluaran yang sudah dikonversi ke Rupiah, lalu membandingkannya dengan batas anggaran (default Rp 10.000.000 atau nominal kustom):

- Sisa Anggaran = Batas Anggaran - Total Pengeluaran IDR
- Persentase Terpakai = (Total Pengeluaran IDR / Batas Anggaran) * 100%

Dari persentase tersebut, server menentukan status peringatan:

* Lebih dari 100%: Status DANGER (anggaran sudah terlampaui/over budget).

* 80% hingga 100%: Status WARNING (peringatan bahwa pengeluaran mendekati batas anggaran).

* Kurang dari 80%: Status SAFE (kondisi keuangan masih aman).

3. Penyimpanan Cadangan Kurs (Fallback Cache):

Server menyimpan nilai kurs terakhir di memori. Jika sewaktu-waktu koneksi internet terganggu saat demo, server tetap dapat melakukan konversi menggunakan nilai cadangan tanpa menyebabkan server error.

5. KODE PROGRAM LENGKAP

5.1 Penjelasan Bagian yang Dikembangkan dari Kode Praktikum

Aplikasi ini merupakan penggabungan sekaligus peningkatan dari materi praktikum sebelumnya:

1. Pengembangan dari Praktikum 2 (API Consumer): Pada Praktikum 2, pemanggilan Public API hanya dilakukan secara terpisah melalui skrip mandiri di terminal. Pada tugas proyek ini, pemanggilan API luar dipindahkan ke dalam arsitektur server, di mana data kurs luar negeri dipadukan dengan database lokal dan rumus anggaran.
2. Pengembangan dari Praktikum 3 (RESTful API Provider): Pada Praktikum 3, server hanya melayani operasi CRUD sederhana pada satu jenis data tanpa pengolahan tambahan. Di proyek ini, kami menambahkan validasi input yang lebih ketat untuk mencegah server berhenti mendadak saat menerima data keliru, fitur filter pencarian melalui query parameter, endpoint ringkasan agregasi data, serta aplikasi client console interaktif terpisah.

5.2 Kode Program Server (server.js)

Berikut adalah kode lengkap server.js (disalin sebagai teks):

```
console.log('============================================================');
console.log('  PROYEK REKAYASA API (SUB-CPMK 4) - TRAVEL EXPENSE API');
console.log('  1. Lidiya - F5512530118');
console.log('  2. Tabita Creaselia Belicia Theona - F5512520086');
console.log('  3. Muhammad Zikri Adam - F5512530117');
console.log('  4. Trista Sheryl Lambei - F5512530106');
console.log('============================================================');

const http = require('http');
const fs = require('fs/promises');
const path = require('path');
const url = require('url');

const PORT = 3000;
const DATA_FILE = path.join(__dirname, 'data', 'expenses.json');
const PUBLIC_API_URL = 'https://open.er-api.com/v6/latest/USD';

const DEFAULT_BUDGET_LIMIT_IDR = 10000000;

// Fallback rates if the external currency API is unreachable
let cachedRates = {
  USD: 1,
  IDR: 16200,
  JPY: 155,
  EUR: 0.92,
  SGD: 1.34,
  MYR: 4.70,
  GBP: 0.78,
  AUD: 1.52
};
let lastFetchedTime = null;

function kirimJSON(res, statusCode, data) {
  const body = JSON.stringify(data, null, 2);
```

```
  res.writeHead(statusCode, {
    'Content-Type': 'application/json; charset=utf-8',
    'Content-Length': Buffer.byteLength(body),
    'Access-Control-Allow-Origin': '*'
  });
  res.end(body);
}

function kirimError(res, statusCode, pesan, detail = null) {
  const payload = {
    sukses: false,
    status: statusCode,
    error: pesan
  };
  if (detail) payload.detail = detail;
  kirimJSON(res, statusCode, payload);
}

function bacaBody(req) {
  return new Promise((resolve, reject) => {
    let body = '';
    req.on('data', (chunk) => {
      body += chunk;
    });
    req.on('end', () => {
      if (!body.trim()) return resolve(null);
      try {
        resolve(JSON.parse(body));
      } catch (err) {
        const errorJSON = new Error('Body request bukan format JSON yang valid');
        errorJSON.statusCode = 400;
        reject(errorJSON);
      }
    });
```

```
    req.on('error', reject);
  });
}

async function dapatkanKursTerbaru() {
  try {
    const response = await fetch(PUBLIC_API_URL, { signal: AbortSignal.timeout(4000) });
    if (response.ok) {
      const data = await response.json();
      if (data && data.rates) {
        cachedRates = data.rates;
        lastFetchedTime = new Date().toISOString();
        return { rates: cachedRates, sumber: 'live_public_api', waktu: lastFetchedTime };
      }
    }
  } catch (err) {
    console.warn('[Peringatan] Gagal memanggil Public API live, beralih ke cache data kurs.');
  }
  return { rates: cachedRates, sumber: 'cache_fallback', waktu: lastFetchedTime ||
'default_rate' };
}

function konversiKeIDR(amount, currency, rates) {
  const mataUang = currency.toUpperCase();
  const rateAsal = rates[mataUang] || 1;
  const rateIDR = rates['IDR'] || 16200;

  // Base rates are USD-anchored: convert source to USD, then USD to IDR
  const nilaiDalamUSD = amount / rateAsal;
  const nilaiDalamIDR = Math.round(nilaiDalamUSD * rateIDR);
  const kursSatuSatuanIDR = Math.round((1 / rateAsal) * rateIDR);

  return {
    nominalIDR: nilaiDalamIDR,
    kursPerSatuanIDR: kursSatuSatuanIDR,
```

```
    formatRupiah: 'Rp ' + nilaiDalamIDR.toLocaleString('id-ID')
  };
}

async function bacaData() {
  try {
    const raw = await fs.readFile(DATA_FILE, 'utf-8');
    return JSON.parse(raw);
  } catch (err) {
    if (err.code === 'ENOENT') {
      await simpanData([]);
      return [];
    }
    throw err;
  }
}

async function simpanData(data) {
  await fs.writeFile(DATA_FILE, JSON.stringify(data, null, 2), 'utf-8');
}

function validasiPayload(body, isUpdate = false) {
  if (!body || typeof body !== 'object' || Array.isArray(body)) {
    return 'Body request wajib berupa objek JSON.';
  }

  const { title, category, amount, currency, date, notes } = body;

  if (!isUpdate || title !== undefined) {
    if (typeof title !== 'string' || title.trim() === '') {
      return 'Field "title" wajib diisi dengan teks.';
    }
  }
```

```
  const kategoriValid = ['akomodasi', 'transportasi', 'konsumsi', 'belanja', 'hiburan',
'lainnya'];
  if (!isUpdate || category !== undefined) {
    if (typeof category !== 'string' || !kategoriValid.includes(category.toLowerCase())) {
      return `Field "category" harus salah satu dari: ${kategoriValid.join(', ')}.`;
    }
  }

  if (!isUpdate || amount !== undefined) {
    if (typeof amount !== 'number' || isNaN(amount) || amount <= 0) {
      return 'Field "amount" wajib berupa angka lebih besar dari 0.';
    }
  }

  const mataUangDidukung = ['USD', 'JPY', 'EUR', 'SGD', 'MYR', 'GBP', 'AUD', 'IDR'];
  if (!isUpdate || currency !== undefined) {
    if (typeof currency !== 'string' || !mataUangDidukung.includes(currency.toUpperCase())) {
      return `Field "currency" harus salah satu dari: ${mataUangDidukung.join(', ')}.`;
    }
  }

  if (!isUpdate || date !== undefined) {
    if (typeof date !== 'string' || !/^\d{4}-\d{2}-\d{2}$/.test(date)) {
      return 'Field "date" wajib format YYYY-MM-DD (contoh: 2026-10-25).';
    }
  }

  if (notes !== undefined && typeof notes !== 'string') {
    return 'Field "notes" harus berupa teks.';
  }

  return null;
}

const server = http.createServer(async (req, res) => {
```

```
  const parsedUrl = url.parse(req.url, true);
  const pathname = parsedUrl.pathname.replace(/\/+$/, '') || '/';
  const method = req.method.toUpperCase();
  const query = parsedUrl.query;

  try {
    if (pathname === '' || pathname === '/') {
      return kirimJSON(res, 200, {
        namaAplikasi: 'Travel Expense Tracker & Currency Converter RESTful API',
        kelompok: 'Kelompok 3 - Rekayasa API (Sub-CPMK 4)',
        anggota: [
          'Lidiya (F5512530118)',
          'Tabita Creaselia Belicia Theona (F5512520086)',
          'Muhammad Zikri Adam (F5512530117)',
          'Trista Sheryl Lambei (F5512530106)'
        ],
        status: 'Server Aktif Berjalan',
        daftarEndpoint: [
          'GET /expenses - Mendapatkan seluruh pengeluaran (bisa filter
?category=...&currency=...)',
          'GET /expenses/summary - Ringkasan total dalam IDR & evaluasi batas anggaran',
          'GET /expenses/:id - Mendapatkan detail satu catatan pengeluaran',
          'POST /expenses - Menambahkan catatan pengeluaran baru',
          'PUT /expenses/:id - Memperbarui catatan pengeluaran',
          'DELETE /expenses/:id - Menghapus catatan pengeluaran'
        ]
      });
    }

    if (pathname === '/expenses/summary' && method === 'GET') {
      const dataExpenses = await bacaData();
      const kursInfo = await dapatkanKursTerbaru();
      const limitBudgetIDR = Number(query.budget) || DEFAULT_BUDGET_LIMIT_IDR;

      let totalAkumulasiIDR = 0;
```

```
      const breakdownKategori = {};

      dataExpenses.forEach((item) => {
        const hasilKonversi = konversiKeIDR(item.amount, item.currency, kursInfo.rates);
        totalAkumulasiIDR += hasilKonversi.nominalIDR;

        if (!breakdownKategori[item.category]) {
          breakdownKategori[item.category] = { totalIDR: 0, count: 0 };
        }
        breakdownKategori[item.category].totalIDR += hasilKonversi.nominalIDR;
        breakdownKategori[item.category].count += 1;
      });

      const sisaAnggaranIDR = limitBudgetIDR - totalAkumulasiIDR;
      const persentasePemakaian = Number(((totalAkumulasiIDR / limitBudgetIDR) *
100).toFixed(2));

      let evaluasiRisiko = 'SAFE: Anggaran Masih Aman Terkendali';
      let tingkatRisiko = 'RENDAH';
      if (persentasePemakaian > 100) {
        evaluasiRisiko = 'DANGER: Pengeluaran Telah Melebihi Batas Anggaran (Over Budget)!';
        tingkatRisiko = 'TINGGI (BAHAYA)';
      } else if (persentasePemakaian >= 80) {
        evaluasiRisiko = 'WARNING: Peringatan Kritis! Lebih dari 80% Kuota Anggaran
Terpakai!';
        tingkatRisiko = 'SEDANG (WASPADA)';
      }

      return kirimJSON(res, 200, {
        sukses: true,
        ringkasanAnggaran: {
          targetAnggaranIDR: limitBudgetIDR,
          formatTargetAnggaran: 'Rp ' + limitBudgetIDR.toLocaleString('id-ID'),
          totalPengeluaranIDR: totalAkumulasiIDR,
          formatTotalPengeluaran: 'Rp ' + totalAkumulasiIDR.toLocaleString('id-ID'),
```

```
          sisaAnggaranIDR: sisaAnggaranIDR,
          formatSisaAnggaran: 'Rp ' + sisaAnggaranIDR.toLocaleString('id-ID'),
          persentasePemakaian: `${persentasePemakaian}%`,
          tingkatRisiko: tingkatRisiko,
          statusEvaluasi: evaluasiRisiko
        },
        rincianKategori: breakdownKategori,
        totalCatatan: dataExpenses.length,
        metadataKurs: {
          sumberAPI: 'ExchangeRate-API (open.er-api.com)',
          statusPemanggilan: kursInfo.sumber,
          kursIDRperUSD: kursInfo.rates['IDR'],
          terakhirDiperbarui: kursInfo.waktu
        }
      });
    }

    if (pathname === '/expenses') {
      const dataExpenses = await bacaData();
      const kursInfo = await dapatkanKursTerbaru();

      if (method === 'GET') {
        let hasil = dataExpenses.map((item) => {
          const konversi = konversiKeIDR(item.amount, item.currency, kursInfo.rates);
          return {
            ...item,
            pengayaanKurs: {
              nominalIDR: konversi.nominalIDR,
              formatRupiah: konversi.formatRupiah,
              kursAcuanKeIDR: `1 ${item.currency} ≈ Rp
${konversi.kursPerSatuanIDR.toLocaleString('id-ID')}`
            }
          };
        });
```

```
        if (query.category) {
          hasil = hasil.filter(
            (item) => item.category.toLowerCase() === query.category.toLowerCase()
          );
        }

        if (query.currency) {
          hasil = hasil.filter(
            (item) => item.currency.toUpperCase() === query.currency.toUpperCase()
          );
        }

        return kirimJSON(res, 200, {
          sukses: true,
          jumlah: hasil.length,
          data: hasil
        });
      }

      if (method === 'POST') {
        const body = await bacaBody(req);
        const pesanErrorValidasi = validasiPayload(body, false);

        if (pesanErrorValidasi) {
          return kirimError(res, 400, pesanErrorValidasi);
        }

        const idBaru = dataExpenses.length > 0 ? Math.max(...dataExpenses.map((d) => d.id)) +
1 : 1;

        const itemBaru = {
          id: idBaru,
          title: body.title.trim(),
          category: body.category.toLowerCase().trim(),
          amount: Number(body.amount),
```

```
          currency: body.currency.toUpperCase().trim(),
          date: body.date,
          notes: body.notes ? body.notes.trim() : ''
        };

        dataExpenses.push(itemBaru);
        await simpanData(dataExpenses);

        const konversi = konversiKeIDR(itemBaru.amount, itemBaru.currency, kursInfo.rates);

        return kirimJSON(res, 201, {
          sukses: true,
          pesan: 'Catatan pengeluaran berhasil ditambahkan.',
          data: {
            ...itemBaru,
            pengayaanKurs: {
              nominalIDR: konversi.nominalIDR,
              formatRupiah: konversi.formatRupiah,
              kursAcuanKeIDR: `1 ${itemBaru.currency} ≈ Rp
${konversi.kursPerSatuanIDR.toLocaleString('id-ID')}`
            }
          }
        });
      }

      return kirimError(res, 405, `Metode ${method} tidak diizinkan pada /expenses`);
    }

    const matchId = pathname.match(/^\/expenses\/(\d+)$/);
    if (matchId) {
      const targetId = parseInt(matchId[1], 10);
      const dataExpenses = await bacaData();
      const itemIndex = dataExpenses.findIndex((d) => d.id === targetId);

      if (method === 'GET') {
```

```
        if (itemIndex === -1) {
          return kirimError(res, 404, `Pengeluaran dengan ID ${targetId} tidak ditemukan.`);
        }

        const kursInfo = await dapatkanKursTerbaru();
        const item = dataExpenses[itemIndex];
        const konversi = konversiKeIDR(item.amount, item.currency, kursInfo.rates);

        return kirimJSON(res, 200, {
          sukses: true,
          data: {
            ...item,
            pengayaanKurs: {
              nominalIDR: konversi.nominalIDR,
              formatRupiah: konversi.formatRupiah,
              kursAcuanKeIDR: `1 ${item.currency} ≈ Rp
${konversi.kursPerSatuanIDR.toLocaleString('id-ID')}`
            }
          }
        });
      }

      if (method === 'PUT') {
        if (itemIndex === -1) {
          return kirimError(res, 404, `Pengeluaran dengan ID ${targetId} tidak ditemukan.`);
        }

        const body = await bacaBody(req);
        const pesanErrorValidasi = validasiPayload(body, true);
        if (pesanErrorValidasi) {
          return kirimError(res, 400, pesanErrorValidasi);
        }

        const itemLama = dataExpenses[itemIndex];
        const itemDiperbarui = {
```

```
          ...itemLama,
          ...(body.title !== undefined && { title: body.title.trim() }),
          ...(body.category !== undefined && { category: body.category.toLowerCase().trim()
}),
          ...(body.amount !== undefined && { amount: Number(body.amount) }),
          ...(body.currency !== undefined && { currency: body.currency.toUpperCase().trim()
}),
          ...(body.date !== undefined && { date: body.date }),
          ...(body.notes !== undefined && { notes: body.notes.trim() })
        };

        dataExpenses[itemIndex] = itemDiperbarui;
        await simpanData(dataExpenses);

        const kursInfo = await dapatkanKursTerbaru();
        const konversi = konversiKeIDR(itemDiperbarui.amount, itemDiperbarui.currency,
kursInfo.rates);

        return kirimJSON(res, 200, {
          sukses: true,
          pesan: `Pengeluaran dengan ID ${targetId} berhasil diperbarui.`,
          data: {
            ...itemDiperbarui,
            pengayaanKurs: {
              nominalIDR: konversi.nominalIDR,
              formatRupiah: konversi.formatRupiah
            }
          }
        });
      }

      if (method === 'DELETE') {
        if (itemIndex === -1) {
          return kirimError(res, 404, `Pengeluaran dengan ID ${targetId} tidak ditemukan.`);
        }
```

```
        const itemDihapus = dataExpenses.splice(itemIndex, 1)[0];
        await simpanData(dataExpenses);

        return kirimJSON(res, 200, {
          sukses: true,
          pesan: `Pengeluaran dengan ID ${targetId} ("${itemDihapus.title}") berhasil
dihapus.`,
          idDihapus: targetId
        });
      }

      return kirimError(res, 405, `Metode ${method} tidak diizinkan pada
/expenses/${targetId}`);
    }

    kirimError(res, 404, `Endpoint ${method} ${pathname} tidak ditemukan.`);
  } catch (error) {
    console.error('Server Internal Error:', error);
    if (error.statusCode === 400) {
      return kirimError(res, 400, error.message);
    }
    kirimError(res, 500, 'Terjadi kesalahan internal pada server.', error.message);
  }
});

server.listen(PORT, () => {
  console.log(`Server aktif berjalan di http://localhost:${PORT}`);
  console.log(`Tekan Ctrl + C untuk mematikan server.`);
});
```

5.3 Kode Program Client Console (client.js)

Berikut adalah kode lengkap client.js (disalin sebagai teks):

```
const readline = require('readline');
const BASE_URL = 'http://localhost:3000';
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
```

```
});
function tanya(promptText) {
  return new Promise((resolve) => {
    rl.question(promptText, (jawaban) => {
      resolve(jawaban.trim());
    });
  });
}
async function requestAPI(endpoint, options = {}) {
  try {
    const url = `${BASE_URL}${endpoint}`;
    const defaultHeaders = {
      'Content-Type': 'application/json',
      'Accept': 'application/json'
    };

    const res = await fetch(url, {
      ...options,
      headers: {
        ...defaultHeaders,
        ...(options.headers || {})
      }
    });

    const data = await res.json().catch(() => null);

    return {
      ok: res.ok,
      status: res.status,
      data: data
    };
  } catch (err) {
    return {
      ok: false,
      status: 0,
      data: {
        error: `Gagal terhubung ke server API (${err.message}). Pastikan server sudah
dijalankan di ${BASE_URL}.`
      }
    };
  }
}

function cetakHeader() {
  console.clear();
```

```
  console.log('==========================================================================');
  console.log('   CLIENT CONSOLE - TRAVEL EXPENSE & CURRENCY CONVERTER CLI');
  console.log('   Rekayasa API (Sub-CPMK 4) - Kelompok 3');
  console.log('   Target API: ' + BASE_URL);
  console.log('==========================================================================\n');
}

async function menuTampilkanSemua() {
  console.log('\n--- DAFTAR SELURUH PENGELUARAN (DIPERKAYA KURS REAL-TIME) ---');
  const res = await requestAPI('/expenses');

  if (!res.ok) {
    console.log(`[Error ${res.status}] ${res.data?.error || 'Gagal mengambil data'}`);
    return;
  }

  const items = res.data.data;
  if (!items || items.length === 0) {
    console.log('Belum ada data pengeluaran tersimpan.');
    return;
  }

  console.log(`Ditemukan ${items.length} catatan pengeluaran:\n`);
  items.forEach((item) => {
    console.log(`[ID: ${item.id}] ${item.title}`);
    console.log(`   Kategori   : ${item.category}`);
    console.log(`   Nominal Asli: ${item.amount} ${item.currency}`);
    console.log(`   Hasil Kurs : ${item.pengayaanKurs.formatRupiah}
(${item.pengayaanKurs.kursAcuanKeIDR})`);
    console.log(`   Tanggal    : ${item.date} | Catatan: ${item.notes || '-'}`);
    console.log('   ' + '-'.repeat(60));
  });
}

async function menuFilterPengeluaran() {
  console.log('\n--- FILTER PENGELUARAN ---');
  console.log('Pilihan filter:');
  console.log('1. Berdasarkan Kategori (akomodasi, transportasi, konsumsi, belanja,
hiburan)');
  console.log('2. Berdasarkan Mata Uang (USD, JPY, EUR, SGD, MYR, dll)');

  const pilihan = await tanya('Pilih jenis filter (1/2): ');
  let endpoint = '/expenses';

  if (pilihan === '1') {
```

```
    const kat = await tanya('Masukkan kategori: ');
    endpoint += `?category=${encodeURIComponent(kat)}`;
  } else if (pilihan === '2') {
    const cur = await tanya('Masukkan kode mata uang (contoh: JPY, USD): ');
    endpoint += `?currency=${encodeURIComponent(cur)}`;
  } else {
    console.log('Pilihan filter tidak valid.');
    return;
  }

  const res = await requestAPI(endpoint);
  if (!res.ok) {
    console.log(`[Error ${res.status}] ${res.data?.error || 'Gagal memfilter data'}`);
    return;
  }

  const items = res.data.data;
  console.log(`\nHasil filter: ditemukan ${items.length} data:\n`);
  items.forEach((item) => {
    console.log(`* [${item.id}] ${item.title} - ${item.amount} ${item.currency} =>
${item.pengayaanKurs.formatRupiah} (${item.category})`);
  });
}

async function menuRingkasanAnggaran() {
  console.log('\n--- RINGKASAN & EVALUASI ANGGARAN (BUDGET GUARD ANALYSIS) ---');
  const inputBudget = await tanya('Masukkan batas anggaran kustom dalam Rp (tekan Enter untuk
default Rp 10.000.000): ');
  const query = inputBudget ? `?budget=${encodeURIComponent(inputBudget)}` : '';

  const res = await requestAPI(`/expenses/summary${query}`);
  if (!res.ok) {
    console.log(`[Error ${res.status}] ${res.data?.error || 'Gagal mengambil ringkasan'}`);
    return;
  }

  const r = res.data.ringkasanAnggaran;
  console.log('\n================ HASIL ANALISIS SERVER ================');
  console.log(`Target Anggaran     : ${r.formatTargetAnggaran}`);
  console.log(`Total Pengeluaran   : ${r.formatTotalPengeluaran}`);
  console.log(`Sisa Anggaran       : ${r.formatSisaAnggaran}`);
  console.log(`Persentase Terpakai : ${r.persentasePemakaian}`);
  console.log(`Tingkat Risiko      : [ ${r.tingkatRisiko} ]`);
  console.log(`Evaluasi Sistem     : ${r.statusEvaluasi}`);
  console.log('=======================================================');
```

```
  console.log('\nRincian Pengeluaran per Kategori:');
  const cats = res.data.rincianKategori;
  for (const [k, v] of Object.entries(cats)) {
    console.log(`  - ${k.padEnd(14)}: Rp ${v.totalIDR.toLocaleString('id-ID')} (${v.count} transaksi)`);
  }

  const meta = res.data.metadataKurs;
  console.log(`\nInformasi Kurs Terkini (USD to IDR): Rp ${meta.kursIDRperUSD?.toLocaleString('id-ID')} [Sumber: ${meta.sumberAPI}]`);
}

async function menuTambahPengeluaran() {
  console.log('\n--- TAMBAH PENGELUARAN BARU (POST) ---');
  const title = await tanya('Judul Pengeluaran (misal: Tiket Museum Kyoto): ');
  const category = await tanya('Kategori (akomodasi/transportasi/konsumsi/belanja/hiburan): ');
  const amountStr = await tanya('Nominal Uang Asing (angka, misal: 1200): ');
  const currency = await tanya('Mata Uang Asing (USD/JPY/EUR/SGD/MYR): ');
  const date = await tanya('Tanggal (YYYY-MM-DD, contoh: 2026-10-26): ');
  const notes = await tanya('Catatan Tambahan (opsional): ');

  const payload = {
    title,
    category,
    amount: Number(amountStr),
    currency,
    date,
    notes
  };

  const res = await requestAPI('/expenses', {
    method: 'POST',
    body: JSON.stringify(payload)
  });

  if (!res.ok) {
    console.log(`\n[Gagal - Status ${res.status}] ${res.data?.error || 'Gagal menambahkan data'}`);
  } else {
    console.log(`\n[Sukses - Status ${res.status}] ${res.data.pesan}`);
    const baru = res.data.data;
    console.log(`ID Baru: ${baru.id} | Dikonversi: ${baru.pengayaanKurs.formatRupiah}`);
  }
```

```
}

async function menuUbahPengeluaran() {
  console.log('\n--- PERBARUI PENGELUARAN (PUT) ---');
  const id = await tanya('Masukkan ID pengeluaran yang ingin diubah: ');

  const cek = await requestAPI(`/expenses/${id}`);
  if (!cek.ok) {
    console.log(`\n[Gagal - Status ${cek.status}] ${cek.data?.error || 'ID tidak
ditemukan'}`);
    return;
  }

  console.log(`Data saat ini: "${cek.data.data.title}" - ${cek.data.data.amount}
${cek.data.data.currency}`);
  console.log('(Tekan Enter langsung jika tidak ingin mengubah field tertentu)\n');

  const title = await tanya('Judul baru: ');
  const category = await tanya('Kategori baru: ');
  const amountStr = await tanya('Nominal baru: ');
  const currency = await tanya('Mata uang baru: ');
  const notes = await tanya('Catatan baru: ');

  const updateBody = {};
  if (title) updateBody.title = title;
  if (category) updateBody.category = category;
  if (amountStr) updateBody.amount = Number(amountStr);
  if (currency) updateBody.currency = currency;
  if (notes) updateBody.notes = notes;

  if (Object.keys(updateBody).length === 0) {
    console.log('Tidak ada data yang diubah.');
    return;
  }

  const res = await requestAPI(`/expenses/${id}`, {
    method: 'PUT',
    body: JSON.stringify(updateBody)
  });

  if (!res.ok) {
    console.log(`\n[Gagal - Status ${res.status}] ${res.data?.error || 'Gagal memperbarui'}`);
  } else {
    console.log(`\n[Sukses - Status ${res.status}] ${res.data.pesan}`);
  }
```

```
}

async function menuHapusPengeluaran() {
  console.log('\n--- HAPUS PENGELUARAN (DELETE) ---');
  const id = await tanya('Masukkan ID pengeluaran yang ingin dihapus: ');

  const konfirmasi = await tanya(`Yakin ingin menghapus ID ${id}? (y/n): `);
  if (konfirmasi.toLowerCase() !== 'y') {
    console.log('Penghapusan dibatalkan.');
    return;
  }

  const res = await requestAPI(`/expenses/${id}`, {
    method: 'DELETE'
  });

  if (!res.ok) {
    console.log(`\n[Gagal - Status ${res.status}] ${res.data?.error || 'Gagal menghapus'}`);
  } else {
    console.log(`\n[Sukses - Status ${res.status}] ${res.data.pesan}`);
  }
}

async function menuUjiKondisiGagal() {
  console.log('\n--- PENGUJIAN SKENARIO GAGAL (MEMBUKTIKAN PENANGANAN ERROR TANPA STACK TRACE)
---');
  console.log('1. Uji Error 400 Bad Request (Payload tidak lengkap / nominal salah)');
  console.log('2. Uji Error 404 Not Found (Mengakses ID yang tidak ada)');
  console.log('3. Uji Error 405 Method Not Allowed');

  const uji = await tanya('Pilih skenario uji (1/2/3): ');

  if (uji === '1') {
    console.log('\nMengirim POST dengan amount negatif dan field title kosong...');
    const res = await requestAPI('/expenses', {
      method: 'POST',
      body: JSON.stringify({ title: '', amount: -500 })
    });
    console.log(`Status HTTP: ${res.status}`);
    console.log(`Pesan Error: ${res.data?.error}`);
    console.log('-> Terbukti: Ditangani dengan elegan (kode status 400, pesan valid, bebas
stack trace).');
  } else if (uji === '2') {
    console.log('\nMengakses GET /expenses/99999 (ID fiktif)...');
    const res = await requestAPI('/expenses/99999');
```

```
    console.log(`Status HTTP: ${res.status}`);
    console.log(`Pesan Error: ${res.data?.error}`);
    console.log('-> Terbukti: Ditangani dengan elegan (kode status 404 Not Found, bebas stack trace).');
  } else if (uji === '3') {
    console.log('\nMengirim PATCH pada /expenses/1 (metode tidak didukung)...');
    const res = await requestAPI('/expenses/1', { method: 'PATCH' });
    console.log(`Status HTTP: ${res.status}`);
    console.log(`Pesan Error: ${res.data?.error}`);
    console.log('-> Terbukti: Ditangani dengan status 405 Method Not Allowed.');
  }
}

async function mainLoop() {
  while (true) {
    cetakHeader();
    console.log('PILIHAN MENU:');
    console.log('1. Tampilkan Semua Pengeluaran (dengan Konversi IDR Real-Time)');
    console.log('2. Filter Pengeluaran (Kategori / Mata Uang)');
    console.log('3. Ringkasan & Analisis Evaluasi Anggaran (Budget Guard Analysis)');
    console.log('4. Tambah Catatan Pengeluaran Baru (POST)');
    console.log('5. Perbarui Catatan Pengeluaran (PUT)');
    console.log('6. Hapus Catatan Pengeluaran (DELETE)');
    console.log('7. Uji Skenario Gagal (Validasi Status 400 & 404 Bebas Stack Trace)');
    console.log('0. Keluar dari Aplikasi');
    console.log('------------------------------------------------------------------------');

    const pilihan = await tanya('Masukkan pilihan Anda (0-7): ');

    switch (pilihan) {
      case '1':
        await menuTampilkanSemua();
        break;
      case '2':
        await menuFilterPengeluaran();
        break;
      case '3':
        await menuRingkasanAnggaran();
        break;
      case '4':
        await menuTambahPengeluaran();
        break;
      case '5':
        await menuUbahPengeluaran();
        break;
```

```
      case '6':
        await menuHapusPengeluaran();
        break;
      case '7':
        await menuUjiKondisiGagal();
        break;
      case '0':
        console.log('\nTerima kasih telah menggunakan Client Console Travel Expense API!');
        rl.close();
        process.exit(0);
      default:
        console.log('\nPilihan tidak valid. Silakan pilih angka 0 - 7.');
    }

    console.log('\nTekan Enter untuk kembali ke menu utama...');
    await tanya('');
  }
}

mainLoop();
```

6. HASIL PENGUJIAN

6.1 Tabel Test Case Pengujian

Untuk membuktikan keandalan sistem, setiap endpoint diuji dengan dua skenario: skenario berhasil (input valid) dan skenario gagal (input salah atau ID tidak ditemukan). Pengujian dilakukan menggunakan aplikasi client API Bruno dengan koleksi 11 skenario uji seperti pada tabel berikut:

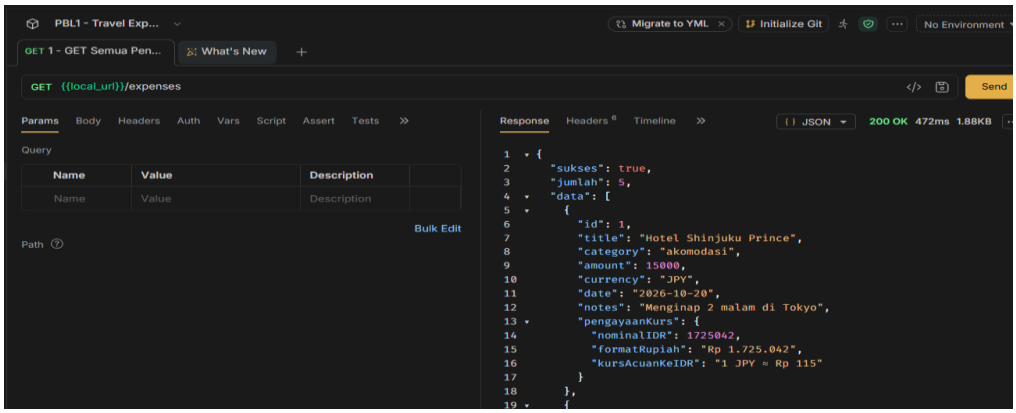
No	Endpoint & Metode	Skenario Uji	Payload / Parameter	Hasil Status Code	Status Hasil
1	GET /expenses	Skenario Berhasil: Mengambil seluruh catatan pengeluaran dengan pengayaan kurs	None	200 OK	BERHASIL (Passed)

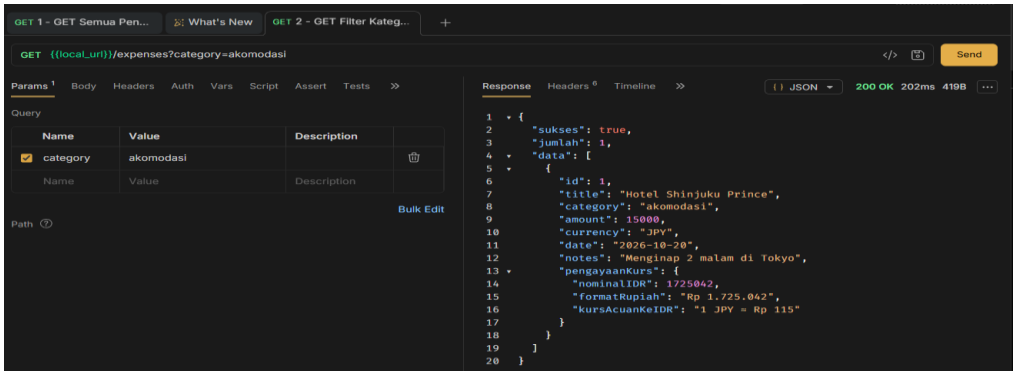
		live IDR			
2	GET /expenses?category=akomodasi	Skenario Berhasil: Filter pengeluaran khusus kategori akomodasi	Query category=akomodasi	200 OK	BERHASIL (Passed)
3	GET /expenses/summary	Skenario Berhasil: Mengkalkulasi total pengeluaran IDR & evaluasi risiko budget	Query budget=10000000	200 OK	BERHASIL (Passed)
4	GET /expenses/1	Skenario Berhasil: Mengambil detail pengeluaran dengan ID valid (ID 1)	Parameter ID: 1	200 OK	BERHASIL (Passed)
5	GET /expenses/9999	Skenario Gagal: Mengakses catatan dengan ID yang tidak terdaftar	Parameter ID: 9999	404 Not Found	BERHASIL (Passed)
6	POST /expenses	Skenario Berhasil: Menambahkan transaksi baru dengan field lengkap dan valid	JSON valid: title, category, amount, currency, date	201 Created	BERHASIL (Passed)
7	POST /expenses	Skenario Gagal: Mengirimkan field title kosong dan	JSON invalid: title: "", amount: -500	400 Bad Request	BERHASIL (Passed)

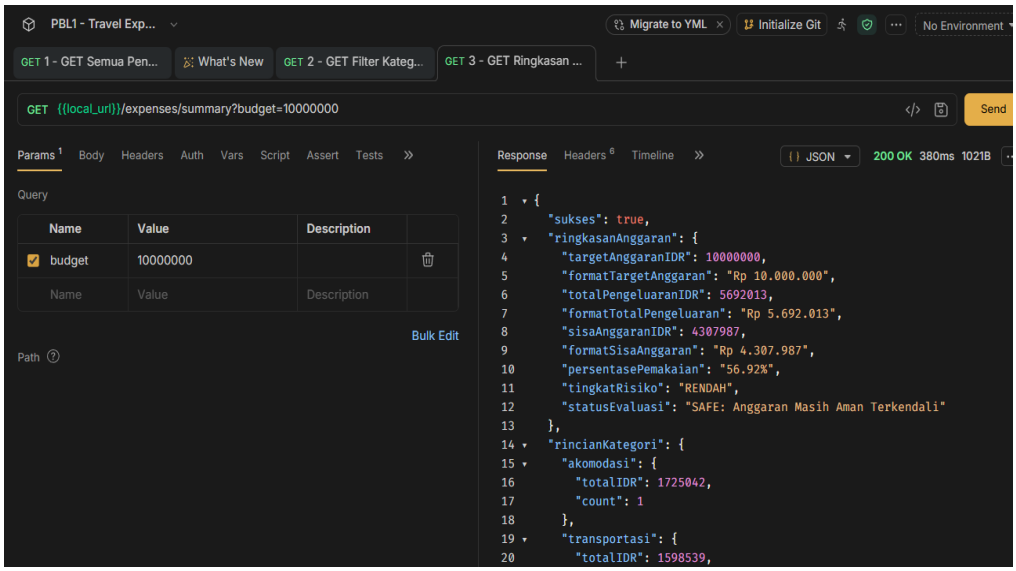
		nominal amount negatif			
8	PUT /expenses/1	Skenario Berhasil: Memperbarui judul dan nominal transaksi ID 1	JSON update: title baru & amount baru	200 OK	BERHASIL (Passed)
9	PUT /expenses/8888	Skenario Gagal: Mencoba memperbarui data pada ID yang tidak ada	Parameter ID: 8888	404 Not Found	BERHASIL (Passed)
10	DELETE /expenses/5	Skenario Berhasil: Menghapus catatan transaksi pada ID valid	Parameter ID: 5	200 OK	BERHASIL (Passed)
11	DELETE /expenses/9999	Skenario Gagal: Menghapus catatan pada ID fiktif yang tidak ditemukan	Parameter ID: 9999	404 Not Found	BERHASIL (Passed)

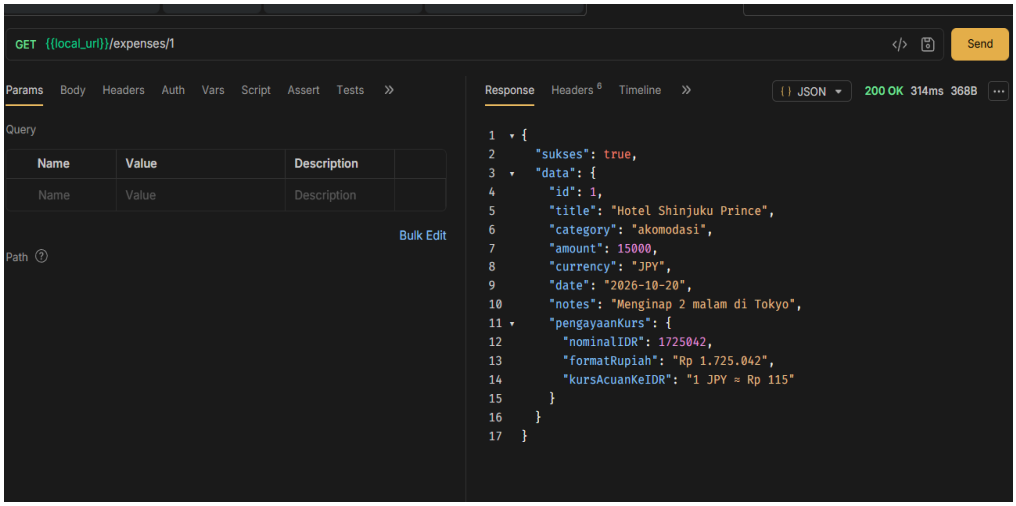
6.2 Bukti Tangkapan Layar Pengujian pada Bruno

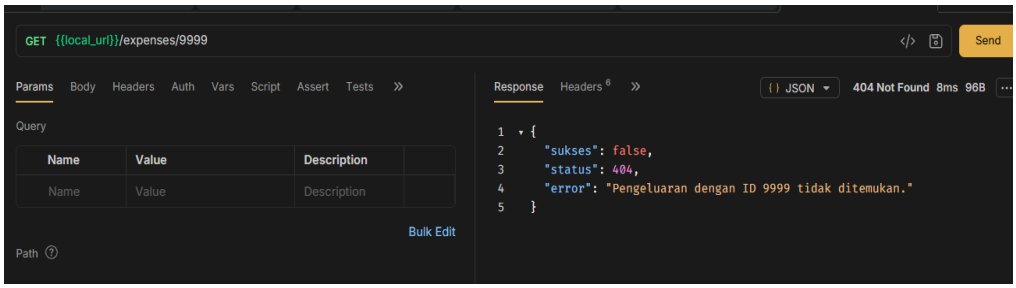
Seluruh pengujian telah dijalankan pada aplikasi Bruno menggunakan koleksi bruno-collection yang kami buat. Berikut adalah tangkapan layar hasil respon untuk masing-masing skenario pengujian:

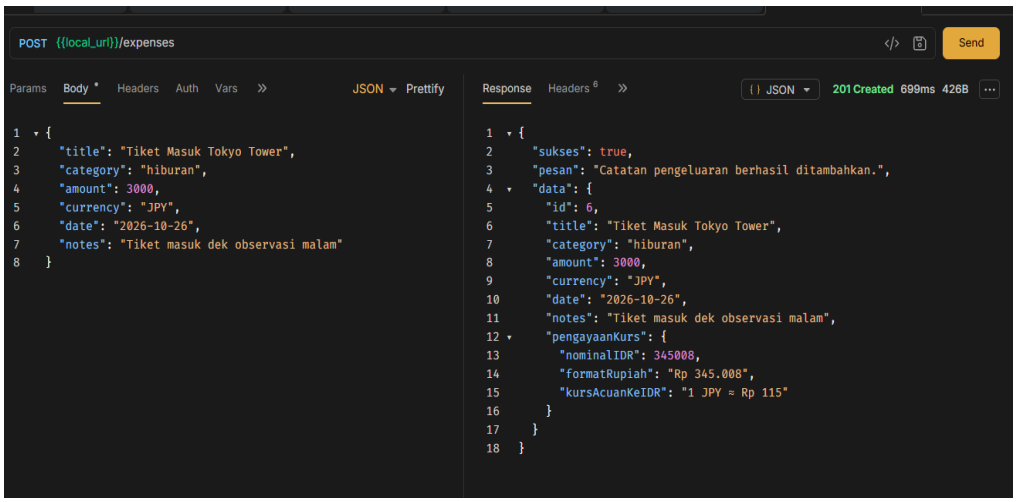

PBL1 - Travel Exp...
Migrate to YML
Initialize Git
No Environment
GET 1 - GET Semua Pen...
What's New
GET {{local_url}}/expenses
Send
Params Body Headers Auth Vars Script Assert Tests
Query
Name Value Description
Bulk Edit
Path
Response Headers 6 Timeline
JSON
200 OK 472ms 1.88KB
{
"sukses": true,
"jumlah": 5,
"data": [
{
"id": 1,
"title": "Hotel Shinjuku Prince",
"category": "akomodasi",
"amount": 15000,
"currency": "JPY",
"date": "2026-10-20",
"notes": "Menginap 2 malam di Tokyo",
"pengayaanKurs": {
"nominalIDR": 1725042,
"formatRupiah": "Rp 1.725.042",
"kursAcuanKeIDR": "1 JPY ≈ Rp 115"
}
},
{

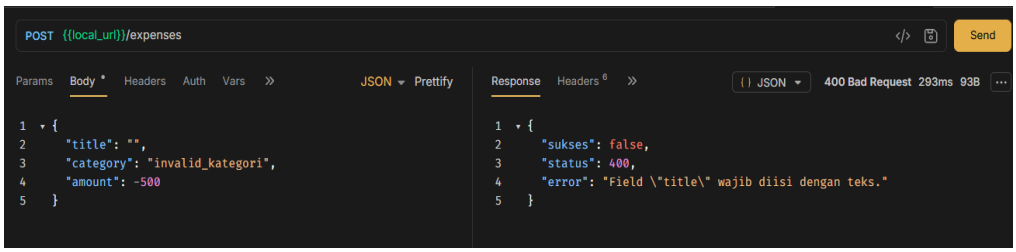

GET 1 - GET Semua Pen...
What's New
GET 2 - GET Filter Kateg...
GET {{local_url}}/expenses?category=akomodasi
Send
Params 1 Body Headers Auth Vars Script Assert Tests
Query
Name Value Description
category akomodasi
Name Value Description
Bulk Edit
Path
Response Headers 6 Timeline
JSON
200 OK 202ms 419B
{
"sukses": true,
"jumlah": 1,
"data": [
{
"id": 1,
"title": "Hotel Shinjuku Prince",
"category": "akomodasi",
"amount": 15000,
"currency": "JPY",
"date": "2026-10-20",
"notes": "Menginap 2 malam di Tokyo",
"pengayaanKurs": {
"nominalIDR": 1725042,
"formatRupiah": "Rp 1.725.042",
"kursAcuanKeIDR": "1 JPY ≈ Rp 115"
}
}
]
}

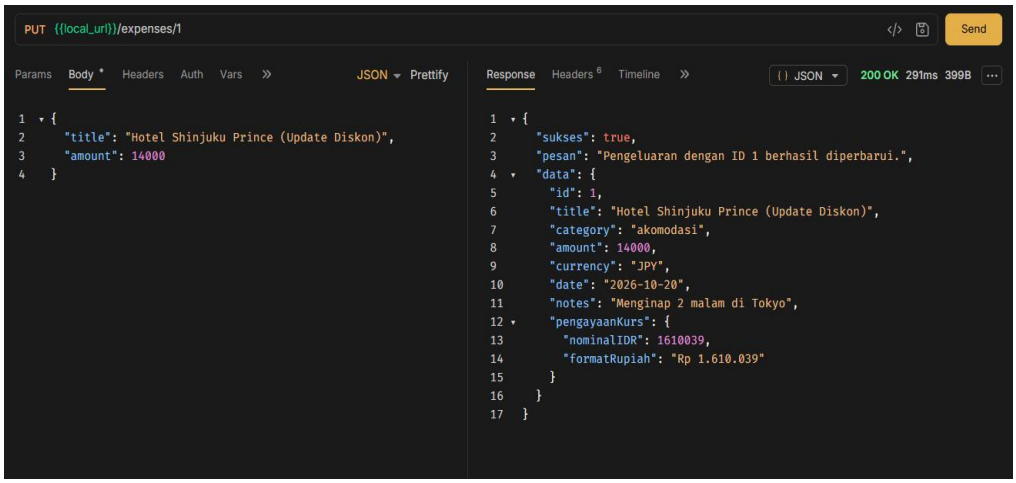

PBL1 - Travel Exp...
Migrate to YML
Initialize Git
No Environment
GET 1 - GET Semua Pen...
What's New
GET 2 - GET Filter Kateg...
GET 3 - GET Ringkasan ...
GET {{local_url}}/expenses/summary?budget=10000000
Send
Params 1
Body
Headers
Auth
Vars
Script
Assert
Tests
Response
Headers 6
Timeline
JSON
200 OK 380ms 1021B
Query
Name
Value
Description
budget
10000000
Bulk Edit
Path
{
"sukses": true,
"ringkasanAnggaran": {
"targetAnggaranIDR": 10000000,
"formatTargetAnggaran": "Rp 10.000.000",
"totalPengeluaranIDR": 5692013,
"formatTotalPengeluaran": "Rp 5.692.013",
"sisaAnggaranIDR": 4307987,
"formatSisaAnggaran": "Rp 4.307.987",
"persentasePemakaian": "56.92%",
"tingkatRisiko": "RENDAH",
"statusEvaluasi": "SAFE: Anggaran Masih Aman Terkendali"
},
"rincianKategori": {
"akomodasi": {
"totalIDR": 1725042,
"count": 1
},
"transportasi": {
"totalIDR": 1598539,

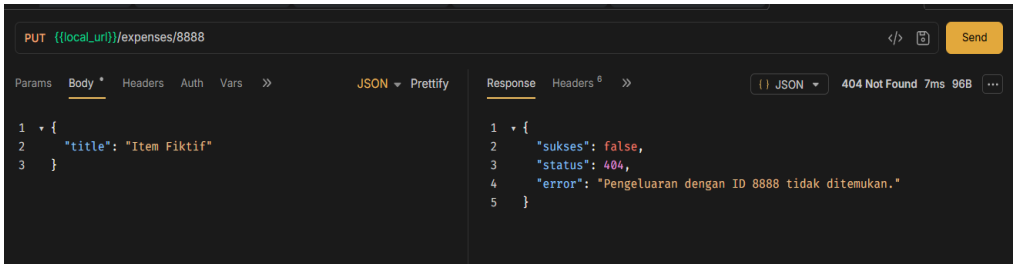

GET {{local_url}}/expenses/1
Send
Params
Body
Headers
Auth
Vars
Script
Assert
Tests
Response
Headers 6
Timeline
JSON
200 OK 314ms 368B
Query
Name
Value
Description
Bulk Edit
Path
{
"sukses": true,
"data": {
"id": 1,
"title": "Hotel Shinjuku Prince",
"category": "akomodasi",
"amount": 15000,
"currency": "JPY",
"date": "2026-10-20",
"notes": "Menginap 2 malam di Tokyo",
"pengayaanKurs": {
"nominalIDR": 1725042,
"formatRupiah": "Rp 1.725.042",
"kursAcuanKeIDR": "1 JPY ≈ Rp 115"
}
}
}

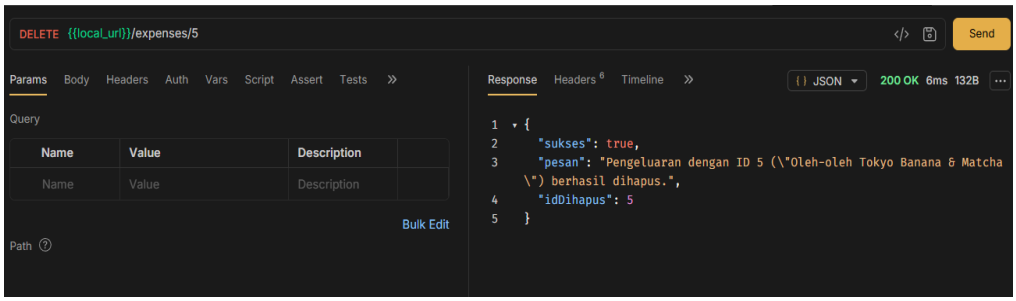

GET {{local_url}}/expenses/9999
Send
Params Body Headers Auth Vars Script Assert Tests
Query
Name Value Description
Name Value Description
Bulk Edit
Path
Response Headers 6
JSON
404 Not Found 8ms 96B
{
"sukses": false,
"status": 404,
"error": "Pengeluaran dengan ID 9999 tidak ditemukan."
}

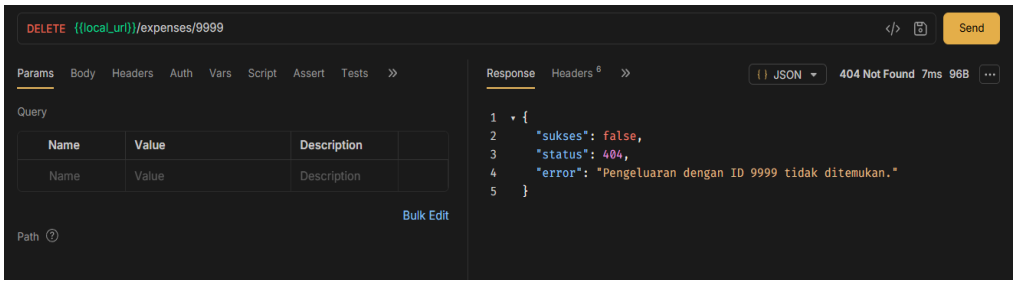

POST {{local_url}}/expenses
Send
Params Body Headers Auth Vars
JSON Prettify
{
"title": "Tiket Masuk Tokyo Tower",
"category": "hiburan",
"amount": 3000,
"currency": "JPY",
"date": "2026-10-26",
"notes": "Tiket masuk dek observasi malam"
}
Response Headers 6
JSON
201 Created 699ms 426B
{
"sukses": true,
"pesan": "Catatan pengeluaran berhasil ditambahkan.",
"data": {
"id": 6,
"title": "Tiket Masuk Tokyo Tower",
"category": "hiburan",
"amount": 3000,
"currency": "JPY",
"date": "2026-10-26",
"notes": "Tiket masuk dek observasi malam",
"pengayaanKurs": {
"nominalIDR": 345008,
"formatRupiah": "Rp 345.008",
"kursAcuanKeIDR": "1 JPY ≈ Rp 115"
}
}
}


POST {{local_url}}/expenses
Send
Params Body Headers Auth Vars
JSON Prettify
{
"title": "",
"category": "invalid_kategori",
"amount": -500
}
Response Headers 6
JSON
400 Bad Request 293ms 93B
{
"sukses": false,
"status": 400,
"error": "Field \"title\" wajib diisi dengan teks."
}


PUT {{local_url}}/expenses/1
Send
Params Body * Headers Auth Vars
JSON Prettify
{
"title": "Hotel Shinjuku Prince (Update Diskon)",
"amount": 14000
}
Response Headers 6 Timeline
JSON 200 OK 291ms 399B
{
"sukses": true,
"pesan": "Pengeluaran dengan ID 1 berhasil diperbarui.",
"data": {
"id": 1,
"title": "Hotel Shinjuku Prince (Update Diskon)",
"category": "akomodasi",
"amount": 14000,
"currency": "JPY",
"date": "2026-10-20",
"notes": "Menginap 2 malam di Tokyo",
"pengayaanKurs": {
"nominalIDR": 1610039,
"formatRupiah": "Rp 1.610.039"
}
}
}


PUT {{local_url}}/expenses/8888
Send
Params Body * Headers Auth Vars
JSON Prettify
{
"title": "Item Fiktif"
}
Response Headers 6
JSON 404 Not Found 7ms 96B
{
"sukses": false,
"status": 404,
"error": "Pengeluaran dengan ID 8888 tidak ditemukan."
}


DELETE {{local_url}}/expenses/5
Send
Params Body Headers Auth Vars Script Assert Tests
Query
Name Value Description
Name Value Description
Bulk Edit
Path
Response Headers 6 Timeline
JSON 200 OK 6ms 132B
{
"sukses": true,
"pesan": "Pengeluaran dengan ID 5 (\"Oleh-oleh Tokyo Banana & Matcha \") berhasil dihapus.",
"idDihapus": 5
}


DELETE {{local_url}}/expenses/9999
Send
Params Body Headers Auth Vars Script Assert Tests
Query
Name Value Description
Name Value Description
Bulk Edit
Path
Response Headers 6
JSON 404 Not Found 7ms 96B
{
"sukses": false,
"status": 404,
"error": "Pengeluaran dengan ID 9999 tidak ditemukan."
}

7. TABEL PEMBAGIAN TUGAS

Setiap anggota kelompok memiliki tanggung jawab spesifik dalam pengerjaan kode program, pengujian, maupun penyusunan dokumentasi laporan. Rincian pembagian tugas tersebut tercantum pada tabel berikut:

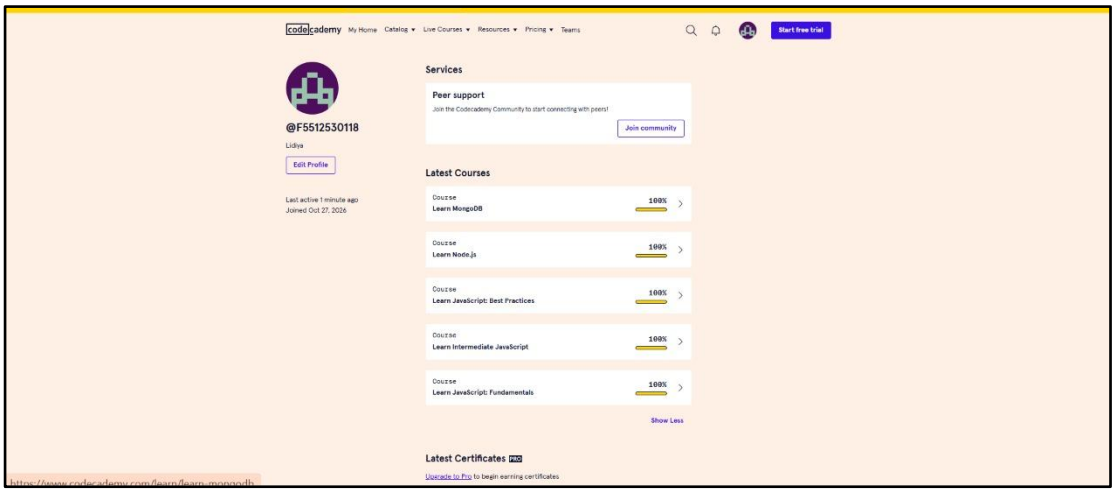
Nama Anggota	**Stambuk (NIM)**	**Bagian Kode / Komponen yang Dikerjakan**	**Tanggung Jawab Teknis**
Lidiya	F5512530118	server.js (Routing CRUD & Identitas Kelompok)	Merancang arsitektur server HTTP native, penanganan route GET /expenses, GET /expenses/:id, status code semantik (200, 404, 405), dan inisialisasi identitas kelompok.
Tabita Creaselia Belicia Theona	F5512520086	server.js (Integrasi Public API & Budget Logic)	Membangun fungsi dapatkanKursTerbaru() via ExchangeRate-API, algoritma konversi kurs silang multi-valas, dan logika analisis evaluasi risiko anggaran pada /expenses/summary.
Muhammad Zikri Adam	F5512530117	server.js (Modul fs Persistensi & Validasi Input)	Mengimplementasikan pembacaan dan penyimpanan data berkas JSON secara asinkron via fs/promises (bacaData, simpanData), serta fungsi validasi payload defensive programming (400 Bad Request).
Trista Sheryl Lambei	F5512530106	client.js (Console CLI) &	Membangun program

		Pengujian Postman	antarmuka terminal interaktif client.js dengan penanganan error ramah tanpa stack trace, menyusun koleksi pengujian Postman, serta mengeksekusi tabel test case.

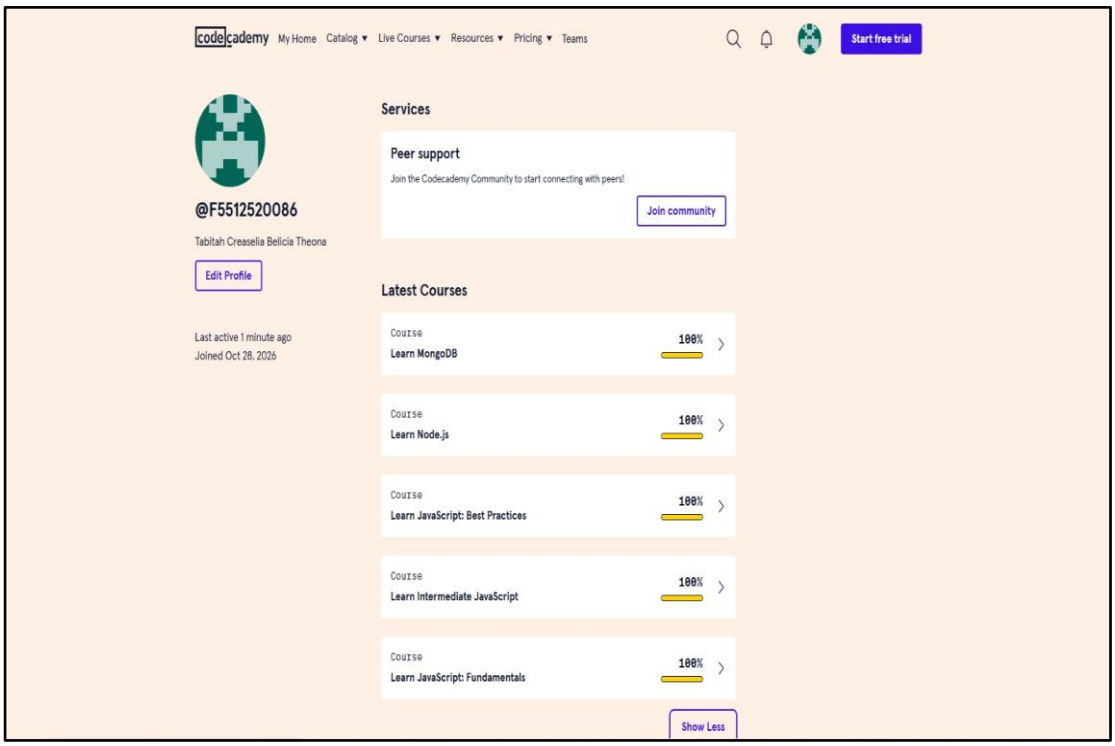
8. LAMPIRAN BUKTI ONLINE COURSE SELURUH ANGGOTA

Sebagai bagian dari Tahap 1 Pembelajaran Mandiri Terstruktur, seluruh anggota kelompok telah menyelesaikan lima course wajib pada platform Codecademy. Bukti tangkapan layar profil akun masing-masing anggota dengan nama lengkap asli dan status penyelesaian 100% terlampir di bawah ini:

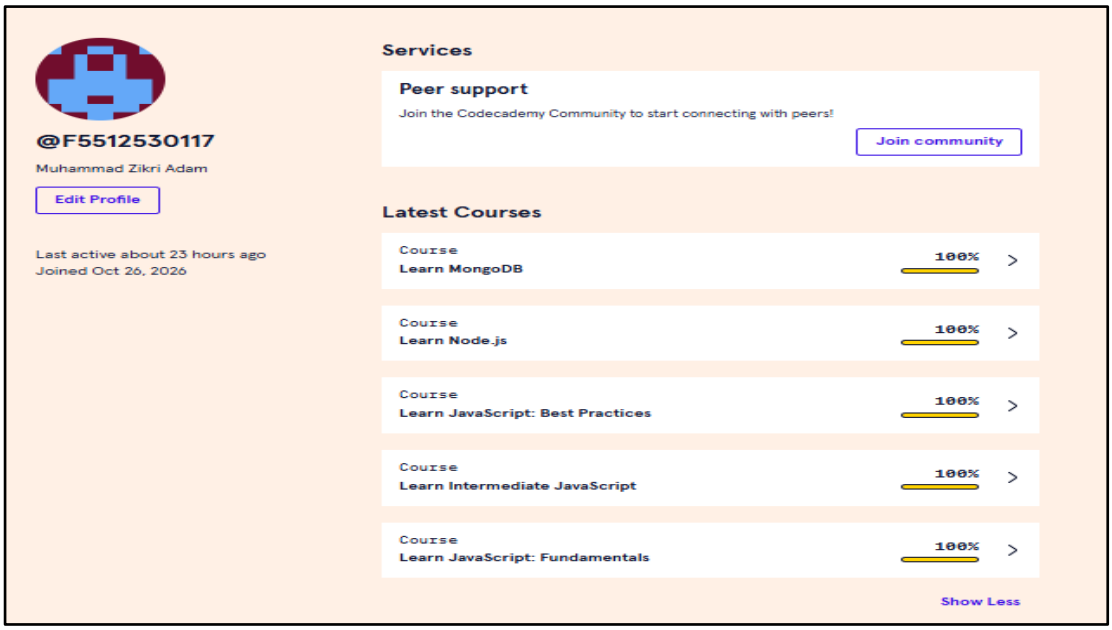
8.1 Bukti Penyelesaian Course: Lidiya (F5512530118)



8.2 Bukti Penyelesaian Course: Tabita Creaselia Belicia Theona (F5512520086)



8.3 Bukti Penyelesaian Course: Muhammad Zikri Adam (F5512530117)



8.4 Bukti Penyelesaian Course: Trista Sheryl Lambei (F5512530106)

